

Programação Orientada aos Objetos

Maio 2026

João Paulo Cascais a110393

João Pontes a111657

Miguel Gonçalves a110879

Grupo 111

João Paulo Cascais || João Pontes || Miguel Gonçalves



1 - Introdução

O DomusControl é uma aplicação de gestão de habitações inteligentes desenvolvida no âmbito da unidade curricular de Programação Orientada aos Objetos. Inspirada em plataformas como o Home Assistant, permite gerir dispositivos conectados, distribuídos por divisões de uma ou mais casas, possibilitando ao utilizador ligar e desligar dispositivos, consultar estatísticas de consumo e gerir os utilizadores associados a cada habitação.

O projeto foi desenvolvido em Java, tirando partido dos princípios fundamentais da Programação Orientada aos Objetos, encapsulamento, herança, abstração e polimorfismo, de forma a criar uma solução extensível e robusta. A arquitetura foi desenhada para permitir a adição de novos tipos de dispositivos sem necessidade de alterar o código existente, respeitando o princípio Open/Closed (aberto a extensão, fechado a modificação).

A aplicação possui uma camada de interação com o utilizador baseada em menus de texto, um sistema de autenticação com distinção entre hosts e guests, estatísticas de consumo em tempo real e mensal, assim como persistência do estado em ficheiro binário.

2. Arquitetura de Classes

2.1 Controlador

O Controlador é a classe central da aplicação, responsável por gerir o estado global do programa.

Contém um objeto Casas (que encapsula um `HashMap<Integer, Casa>`) para armazenar as casas, um objeto Utilizadores para os utilizadores, e um `long tempoAtual` que representa o tempo simulado da aplicação em minutos. A decisão de encapsular as coleções em classes próprias (Casas, Utilizadores), em vez de manter os `HashMaps` diretamente no Controlador, permite que cada coleção seja responsável pela sua própria lógica. Operações como "casa que mais consome" ou "top 3 divisões com mais dispositivos" residem na classe Casas, evitando que o Controlador acumule responsabilidades que não lhe pertencem.

A escolha de `HashMap` para indexar casas e utilizadores por ID permite acesso direto em $O(1)$, o que é uma mais-valia dado que a maioria das operações requer aceder a uma entidade específica pelo seu identificador.

O Controlador é também responsável pela persistência do estado, implementando os métodos `guardaEstado()` e `carregaEstado()`, que serializam e desserializam o objeto para ficheiro binário.

2.2 Casa

A classe Casa representa uma habitação no sistema. Contém um `HashMap<Integer, Divisao>` para as suas divisões, um `idHost` que identifica o proprietário, e uma `List<Integer> idGuests` que armazena os IDs dos utilizadores convidados.

A distinção entre host e guest foi uma decisão de design importante. O host tem controlo total sobre a casa, adicionar e remover divisões e dispositivos, gerir guests, enquanto os guests apenas interagem com os dispositivos. Esta distinção é feita através do `idHost` guardado na Casa e comparado com o `idUtilizador` do utilizador autenticado, sendo formalizada pelo enum `Permissoes` (HOST, GUEST, NADA).

2.3 Divisão

A classe Divisao representa uma divisão de uma casa. Contém uma `List<ADomusSimples>` com os dispositivos associados.

A decisão de usar uma `List` em vez de um `HashMap` deve-se ao facto de os dispositivos numa divisão serem tipicamente iterados na totalidade, para listar, ligar todos, ou calcular consumo, tornando a lista mais adequada que um mapa. Sendo uma divisão tipicamente composta por poucos dispositivos, a perda de acesso direto por ID não é significativa.

2.4 Utilizador e Utilizadores

A classe Utilizador representa um utilizador do sistema com os seus dados pessoais e uma `List<Integer> idCasas` que faz referência às casas a que tem acesso. A classe Utilizadores encapsula um `HashMap<Integer, Utilizador>` e disponibiliza métodos de autenticação através do `efetuaLogin()`.

A decisão de guardar apenas os IDs das casas no Utilizador, em vez das casas completas, evita redundância de dados: as casas estão centralizadas no Controlador, e o Utilizador apenas mantém referências por ID. Esta abordagem facilita também a manutenção da consistência. Uma alteração a uma casa propaga-se automaticamente sem necessidade de sincronização entre cópias.

2.5 Dispositivos

A hierarquia de dispositivos foi desenhada para ser extensível.

A classe abstrata `ADomusSimples` define o comportamento base de qualquer dispositivo: identificador, marca, modelo, consumo, estado e métricas de utilização (`numAtivacoes`, `tempoAcumulado`, `momentoLigado`). A classe abstrata `ADomusComplexo` estende `ADomusSimples` e adiciona os modos ECO e BOOST, servindo de base para dispositivos com mais estados.

As interfaces `IDomusSimples` e `IDomusComplexo` definem os contratos que as classes abstratas implementam, garantindo que qualquer novo dispositivo respeita os métodos essenciais. A interface `IDomusAC` exemplifica como comportamentos específicos podem ser definidos por contrato, declarando `setModoAC()` e `setTemperaturaAC()`, sem comprometer a hierarquia base.

Esta arquitetura respeita o princípio Open/Closed: para adicionar um novo dispositivo (por exemplo, uma `DomusTomada`), bastaria criar uma classe `DomusTomada` que extende `ADomusSimples`, sem qualquer modificação ao Controlador, à Casa, à Divisão ou aos restantes dispositivos.

O campo `nivel` foi adicionado à `ADomusSimples` para representar de forma genérica a intensidade de um dispositivo como luminosidade numa lâmpada, volume numa coluna, abertura numa persiana, evitando duplicação de código nas subclasses. O uso de enums (`Estados` com ON, OFF, ECO, BOOST; `Permissoes` com HOST, GUEST, NADA) em vez de constantes inteiras ou strings garante segurança de tipos e impossibilita a atribuição de estados inválidos por design.

O tempo de utilização é calculado através do `momentoLigado`: quando um dispositivo é ligado, guarda-se o `tempoAtual` do Controlador; quando é desligado, a diferença é acumulada no `tempoAcumulado`. O `getTempoTotal()` inclui o período atual caso o dispositivo esteja ligado, fornecendo um valor sempre atualizado.

A hierarquia de menus segue uma estrutura em árvore que reflete a navegação da aplicação.

Esta abordagem garante que cada menu é responsável apenas pela sua camada de navegação, delegando ao Controlador toda a lógica de negócio — uma separação clara entre apresentação e lógica que facilita a manutenção e a evolução do código.

```

classDiagram
    class AppDomusControl
    class Controlador {
        utilizadores : Utilizadores
        casas : Casas
        temperatura : long
        casar : Utilizador
        idUtilizador : Set<Integer>
    }
    class Utilizadores {
        <<abstract>>
        utilizadores : HashMap<Integer, Utilizador>
    }
    class Cakes {
        cake : HashMap<Integer, Cakes>
    }
    class Utilizador {
        nome : String
        id : int
        email : String
        password : String
        idUtilizador : int
        idCakes : List<Integer>
    }
    class Casas {
        idCakes : HashMap<Integer, Divisao>
        idCakes : int
        idCakes : String
        idCakes : String
        idCakes : List<Integer>
    }
    class Divisao {
        idCakes : String
        idCakes : String
        idCakes : List<Integer>
    }
    class Menus {
        <<abstract>>
        menuMenu : String
        idCakes : Scanner
    }
    class MenuAuth
    class MainMenu
    class MenuGestaCakes
    class MenuGestaCasa
    class MenuGestaDiv
    class MenuGestaDive
    class MenuGestaAC
    class MenuGestaDts
    class IDomusCompleto {
        <<interface>>
        idCakes()
        idCakes()
    }
    class ADomusCompleto {
        <<abstract>>
    }
    class ADomusSimple {
        <<abstract>>
        idCapto : int
        consumoOgto : double
        consumoAtual : double
        entregaAtual : double
        Estado : String
        marca : String
        modelo : String
        numArteses : int
        tempoAcumulado : long
        tempoAcumulado : long
        momentoLigado : long
        nivel : double
    }
    class DomusAmpla {
        temperaturaAC : int
        idCakes : boolean
    }
    class DomusAC {
        temperaturaAC : int
        temperaturaAC : boolean
    }
    class DomusACInterface {
        <<interface>>
        idCakes()
        idCakes()
    }
    class DomusGradas
    class DomusGrador {
        idArteses : int
        idArteses : int
    }
    class DomusSoundSystem {
        idArteses : String
    }
    class Estados {
        estado : ON, OFF, ECO, BOOST
    }
    AppDomusControl --> Controlador
    Controlador --> Utilizadores
    Controlador --> Cakes
    Controlador --> Utilizador
    Controlador --> Casas
    Controlador --> Divisao
    Controlador --> Menus
    Controlador --> IDomusCompleto
    Controlador --> ADomusCompleto
    Controlador --> ADomusSimple
    Controlador --> DomusAmpla
    Controlador --> DomusAC
    Controlador --> DomusACInterface
    Controlador --> DomusGradas
    Controlador --> DomusGrador
    Controlador --> DomusSoundSystem
    Controlador --> Estados
    Utilizadores --> Cakes
    Utilizadores --> Utilizador
    Cakes --> Utilizador
    Casas --> Utilizador
    Casas --> Divisao
    Utilizador --> Casas
    Utilizador --> Divisao
    Casas --> Divisao
    Menus --> MenuAuth
    Menus --> MainMenu
    Menus --> MenuGestaCakes
    Menus --> MenuGestaCasa
    Menus --> MenuGestaDiv
    Menus --> MenuGestaDive
    MenuGestaCakes --> MenuGestaAC
    MenuGestaCakes --> MenuGestaDts
    MenuGestaCasa --> MenuGestaAC
    MenuGestaCasa --> MenuGestaDts
    IDomusCompleto ..> ADomusCompleto
    ADomusCompleto ..> ADomusSimple
    ADomusSimple ..> DomusAmpla
    ADomusSimple ..> DomusAC
    ADomusSimple ..> DomusACInterface
    ADomusSimple ..> DomusGradas
    ADomusSimple ..> DomusGrador
    ADomusSimple ..> DomusSoundSystem
    ADomusSimple ..> Estados
    DomusAmpla ..> DomusAC
    DomusAC ..> DomusACInterface
    DomusACInterface ..> DomusGradas
    DomusACInterface ..> DomusGrador
    DomusACInterface ..> DomusSoundSystem
    DomusGradas ..> DomusGrador
    DomusGrador ..> DomusSoundSystem
    DomusSoundSystem ..> Estados
    Estados ..> Estados
  
```

4. Descrição da Aplicação

4.1 Autenticação

O programa arranca carregando automaticamente o estado guardado em ficheiro binário. Caso não exista ficheiro, é inicializado com dados de teste. O utilizador é apresentado com o menu de autenticação onde pode fazer login ou criar conta.

```
==== Welcome to the Authentication Menu. For any inconveniences, contact our support team.====

1- Login
2- Create Account
3- Exit

1
Email:
joao@gmail.com
Password:
1234
Login as been successful. Welcome back joao!

==== Welcome to the Main Menu! DomusControl provides you with the most advanced house-management interface. Enjoy!====

1 - List all your corresponding proprieties.
2 - List all your own proprieties.
3 - List all the proprieties you've been invited to.
4 - Save state.

0 - Go back.
99 - Exit.
```

4.2 Gestão de Casas

Após o login, o utilizador acede ao menu principal onde pode listar e gerir as suas casas. É possível adicionar e remover casas da base de dados, bem como aceder à gestão individual de cada casa.

```
ID do Host: 1
ID da Casa: 1
Morada: Rua A
Nome Associado: Casa Principal
Lista de Divisões Associadas:
{1=Sala Com ID atribuído: 1
Lista de dispositivos associados: [
| ID: 1
| Marca: Philips
| Modelo: HUE
| Consumo: 10.0wh
| Consumo Atual: 10.0wh/h
| Estado: ON
| Intensidade: 50.0%| Cor: 2700K,
| ID: 2
| Marca: Ikea
| Modelo: Tradfri
| Consumo: 8.0wh
| Consumo Atual: 8.0wh/h
| Estado: OFF
| Intensidade: 30.0%,
| ID: 3
| Marca: Xiaomi
| Modelo: Yeelight
| Consumo: 9.0wh
| Consumo Atual: 9.0wh/h
| Estado: ON
| Intensidade: 70.0%| Cor: 4000K]
, 2=Quarto Com ID atribuído: 2
Lista de dispositivos associados: [
| ID: 4
| Marca: Philips
| Modelo: HUE2
| Consumo: 12.0wh
| Consumo Atual: 12.0wh/h
| Estado: OFF
| Intensidade: 80.0%| Cor: 3000K]
```

4.3 Gestão de uma Casa

Dentro de uma casa específica, o utilizador pode consultar o consumo em tempo real, o consumo mensal, as três divisões com mais dispositivos, bem como gerir os utilizadores convidados, adicionando ou removendo guests.

```
==== Single Propriety Management Menu.====
1 - List all users.
2 - List guests.
3 - List hosts
4 - Consult real time propriety consumption.
5 - Consult monthly propriety consumption
6 - Show the top 3 divisions with the most devices.
7 - Remove user.
8 - Add user.
9 - Implement user permissions.
12 - Add division.
13 - Remove division.
14 - Division management.
15 - Show the top 3 most consuming devices by time.
16 - Show the top 3 most consuming devices by activations.
0 - Go back.
99 - Exit.
|
```

```
==== Single Propriety Management Menu.====
1 - List all users.
2 - List guests.
3 - List hosts
4 - Consult real time propriety consumption.
5 - Consult monthly propriety consumption
6 - Show the top 3 divisions with the most devices.
7 - Remove user.
8 - Add user.
9 - Implement user permissions.
12 - Add division.
13 - Remove division.
14 - Division management.
15 - Show the top 3 most consuming devices by time.
16 - Show the top 3 most consuming devices by activations.
0 - Go back.
99 - Exit.
4
Current propriety consumption: 19.0 Wh
```

```
==== Single Propriety Management Menu.====
1 - List all users.
2 - List guests.
3 - List hosts
4 - Consult real time propriety consumption.
5 - Consult monthly propriety consumption
6 - Show the top 3 divisions with the most devices.
7 - Remove user.
8 - Add user.
9 - Implement user permissions.
12 - Add division.
13 - Remove division.
14 - Division management.
15 - Show the top 3 most consuming devices by time.
16 - Show the top 3 most consuming devices by activations.
0 - Go back.
99 - Exit.
5
Consumo mensal da casa: 86.33333333333333 Wh
```

```
6
Top 3 most consuming divisions: [Sala Com ID atribuido: 1
Lista de dispositivos associados: [
| ID: 1
| Marca: Philips
| Modelo: HUE
| Consumo: 10.0Wh
| Consumo Atual: 10.0Wh/h
| Estado: ON
| Intensidade: 50.0%| Cor: 2700K,
| ID: 2
| Marca: Ikea
| Modelo: Tradfri
| Consumo: 8.0Wh
| Consumo Atual: 8.0Wh/h
| Estado: OFF
| Intensidade: 30.0%,
| ID: 3
| Marca: Xiaomi
| Modelo: Yeelight
| Consumo: 9.0Wh
| Consumo Atual: 9.0Wh/h
| Estado: ON
| Intensidade: 70.0%| Cor: 4000K]
```

```
, Cozinha Com ID atribuido: 3
Lista de dispositivos associados: [
| ID: 5
| Marca: Osram
| Modelo: Smart+
| Consumo: 7.0Wh
| Consumo Atual: 7.0Wh/h
| Estado: OFF
| Intensidade: 40.0%,
| ID: 6
| Marca: Ikea
| Modelo: Tradfri2
| Consumo: 6.0Wh
| Consumo Atual: 6.0Wh/h
| Estado: OFF
| Intensidade: 20.0%]
, Quarto Com ID atribuido: 2
Lista de dispositivos associados: [
| ID: 4
| Marca: Philips
| Modelo: HUE2
| Consumo: 12.0Wh
| Consumo Atual: 12.0Wh/h
| Estado: OFF
| Intensidade: 80.0%| Cor: 3000K]
```

4.4 Gestão de Divisões

O utilizador pode navegar pelas divisões da casa, listar os dispositivos de cada divisão e adicionar ou remover divisões.

```
==== Division Management Menu.====  
  
1 - Select division.  
2 - Shutdown Mode.  
  
0 - Go back.  
99 - Exit.
```

4.5 Gestão de Dispositivos

Dentro de uma divisão, o utilizador pode listar todos os dispositivos, consultar o dispositivo de maior consumo, ver as estatísticas da divisão (top 3 dispositivos por tempo de utilização e por número de ativações), ligar e desligar dispositivos individualmente, e ativar os modos BOOST e ECO em dispositivos complexos. Existe ainda um submenu dedicado à adição de novos dispositivos (lâmpada, ar condicionado, persianas, regador, sistema de som) e um submenu específico para configuração do ar condicionado, onde é possível alternar entre os modos de aquecimento, arrefecimento e ventilação.

```
==== Single Division Management Menu.====
```

```
1 - List all your devices.  
2 - Consult the top consuming device.  
3 - Consult division stats.  
4 - Turn on a device.  
5 - Turn off a device.  
6 - Boost a device.  
7 - Eco a device.  
8 - Add Device.  
9 - AC Management.  
10 - Set Device Level  
0 - Go back.  
99 - Exit.
```

```
1  
[  
| ID: 1  
| Marca: Philips  
| Modelo: HUE  
| Consumo: 10.0Wh  
| Consumo Atual: 10.0Wh/h  
| Estado: ON  
| Intensidade: 50.0%| Cor: 2700K,  
| ID: 2  
| Marca: Ikea  
| Modelo: Tradfri  
| Consumo: 8.0Wh  
| Consumo Atual: 8.0Wh/h  
| Estado: OFF  
| Intensidade: 30.0%,  
| ID: 3  
| Marca: Xiaomi  
| Modelo: Yeelight  
| Consumo: 9.0Wh  
| Consumo Atual: 9.0Wh/h  
| Estado: ON  
| Intensidade: 70.0%| Cor: 4000K]
```

4.6 Estatísticas

A aplicação disponibiliza um conjunto de estatísticas relevantes:

- **Consumo real:** Soma do consumo atual de todos os dispositivos ligados na casa, tendo em conta o multiplicador de consumo consoante o estado (ON, ECO, BOOST)
- **Consumo mensal:** Calculado com base no tempo acumulado de utilização de cada dispositivo
- **Top 3 dispositivos por tempo:** Os três dispositivos que estiveram ligados mais tempo
- **Top 3 dispositivos por ativações:** Os três dispositivos que foram ligados mais vezes
- **Top 3 divisões com mais dispositivos:** As três divisões com maior número de dispositivos associados

```
==== Single Propriety Management Menu.====
1 - List all users.
2 - List guests.
3 - List hosts
4 - Consult real time propriety consumption.
5 - Consult monthly propriety consumption
6 - Show the top 3 divisions with the most devices.
7 - Remove user.
8 - Add user.
9 - Implement user permissions.
12 - Add division.
13 - Remove division.
14 - Division management.
15 - Show the top 3 most consuming devices by time.
16 - Show the top 3 most consuming devices by activations.
0 - Go back.
99 - Exit.
4
Current propriety consumption: 19.0 Wh
```

```
==== Single Propriety Management Menu.====
1 - List all users.
2 - List guests.
3 - List hosts
4 - Consult real time propriety consumption.
5 - Consult monthly propriety consumption
6 - Show the top 3 divisions with the most devices.
7 - Remove user.
8 - Add user.
9 - Implement user permissions.
12 - Add division.
13 - Remove division.
14 - Division management.
15 - Show the top 3 most consuming devices by time.
16 - Show the top 3 most consuming devices by activations.
0 - Go back.
99 - Exit.
5
Consumo mensal da casa: 86.33333333333333 Wh
```

```
Top 3 por tempo: [
| ID: 1
| Marca: Philips
| Modelo: HUE
| Consumo: 10.0Wh
| Consumo Atual: 10.0Wh/h
| Estado: ON
| Intensidade: 50.0%| Cor: 2700K,
| ID: 2
| Marca: Ikea
| Modelo: Tradfri
| Consumo: 8.0Wh
| Consumo Atual: 8.0Wh/h
| Estado: OFF
| Intensidade: 30.0%,
| ID: 3
| Marca: Xiaomi
| Modelo: Yeelight
| Consumo: 9.0Wh
| Consumo Atual: 9.0Wh/h
| Estado: ON
| Intensidade: 70.0%| Cor: 4000K]
```

```
Top 3 por ativações: [
| ID: 1
| Marca: Philips
| Modelo: HUE
| Consumo: 10.0Wh
| Consumo Atual: 10.0Wh/h
| Estado: ON
| Intensidade: 50.0%| Cor: 2700K,
| ID: 3
| Marca: Xiaomi
| Modelo: Yeelight
| Consumo: 9.0Wh
| Consumo Atual: 9.0Wh/h
| Estado: ON
| Intensidade: 70.0%| Cor: 4000K,
| ID: 2
| Marca: Ikea
| Modelo: Tradfri
| Consumo: 8.0Wh
| Consumo Atual: 8.0Wh/h
| Estado: OFF
| Intensidade: 30.0%]
```

4.7 Persistência

O estado da aplicação pode ser guardado em qualquer momento através da opção "Save state" no menu principal. O ficheiro gerado é binário (estado.dat) e é carregado automaticamente no próximo arranque da aplicação, garantindo que toda a informação (casas, divisões, dispositivos e utilizadores) é preservada entre sessões.

```
==== Welcome to the Main Menu! DomusControl provides you with the most advanced house-management interface. Enjoy!====  
1 - List all your corresponding proprieties.  
2 - List all your own proprieties.  
3 - List all the proprieties you've been invited to.  
4 - Save state.  
0 - Go back.  
99 - Exit.  
4  
Estado guardado.
```

4.8 Encapsulamento e Cópia Defensiva

Para garantir o encapsulamento das estruturas internas, todas as classes que manipulam coleções (Casa, Casas, Divisao, Utilizadores) implementam cópia defensiva. Os construtores de cópia, getters e setters criam clones dos objetos em vez de devolverem ou guardarem referências diretas, impedindo que código externo modifique o estado interno de uma classe sem passar pelos métodos públicos. Os dispositivos (ADomusSimples e subclasses) implementam um método clone() abstrato que cada subclasse concretiza, permitindo que cópias polimórficas funcionem corretamente em toda a hierarquia.

5. Conclusão

O desenvolvimento do DomusControl permitiu aplicar de forma prática os conceitos fundamentais da Programação Orientada aos Objetos. A hierarquia de classes dos dispositivos, com ADomusSimples e ADomusComplexo como classes abstratas e as interfaces IDomusSimples, IDomusComplexo e IDomusAC a definir contratos claros, garante que novos tipos de dispositivos podem ser adicionados sem alterar o código existente. A separação entre a camada de interação com o utilizador (menus) e a lógica de negócio (Controlador) torna o código mais organizado e fácil de manter.

Durante o desenvolvimento, enfrentámos alguns desafios. A simulação da passagem do tempo para o cálculo correto do consumo acumulado dos dispositivos exigiu uma reflexão cuidadosa sobre como modelar o estado temporal sem complicar a interface das classes. A necessidade de garantir o encapsulamento através do uso sistemático de cópias defensivas — nos construtores, nos getters e nos setters — exigiu disciplina ao longo de todas as classes que manipulam coleções.

O projeto cumpre integralmente os requisitos base de gestão de entidades e os requisitos de estatísticas e persistência em ficheiro binário, implementando ainda funcionalidades adicionais como a distinção entre hosts e guests e o cálculo de consumo mensal baseado no tempo real de utilização dos dispositivos.

Como trabalho futuro, identificamos a implementação dos requisitos de nível superior previstos no enunciado: automações dependentes de condições externas e temporais, escalonamentos baseados em horários, e cenários que atuem sobre múltiplos dispositivos simultaneamente (como "Sair de casa", "Jantar com amigos", "Deitar" ou "Acordar"). A arquitetura atual, com o Controlador como ponto central de entrada e a hierarquia abstrata de dispositivos, está preparada para acomodar estas extensões sem necessidade de alterações estruturais profundas — bastará introduzir novas classes (Automacao, Escalonamento, Cenario) e associá-las às casas. Adicionalmente, a sugestão automática de automações com base nos padrões de utilização do utilizador, prevista no requisito mais avançado, surge como uma extensão natural a partir das estatísticas já recolhidas.

Como trabalho futuro, identificamos a possibilidade de tornar o método executa() abstrato na classe Menus, o que exigiria refatorar as assinaturas dos métodos, e a implementação das automações, escalonamentos e cenários previstos nos requisitos de nível superior.